



List<E>接口与ArrayList<E>

北京理工大学计算机学院
金旭亮

List概述

- List通过索引来访问元素，看成是一种可“动态改变大小的数组”，其接口包容很多的成员，如下图所示：

I List		
m	size()	int
m	isEmpty()	boolean
m	contains(Object)	boolean
m	iterator()	Iterator<E>
m	toArray()	Object[]
m	toArray(T[])	T[]
m	add(E)	boolean
m	remove(Object)	boolean
m	containsAll(Collection<?>)	boolean
m	addAll(Collection<? extends E>)	boolean
m	addAll(int, Collection<? extends E>)	boolean
m	removeAll(Collection<?>)	boolean
m	retainAll(Collection<?>)	boolean
m	replaceAll(UnaryOperator<E>)	void
m	sort(Comparator<? super E>)	void
m	clear()	void
m	equals(Object)	boolean
m	hashCode()	int

m	get(int)	E
m	set(int, E)	E
m	add(int, E)	void
m	remove(int)	E
m	indexOf(Object)	int
m	lastIndexOf(Object)	int
m	listIterator()	ListIterator<E>
m	listIterator(int)	ListIterator<E>
m	subList(int, int)	List<E>
m	splitter()	Splitter<E>

m	of()	List<E>
m	of(E)	List<E>
m	of(E, E)	List<E>
m	of(E, E, E)	List<E>
m	of(E, E, E, E)	List<E>
m	of(E, E, E, E, E)	List<E>
m	of(E, E, E, E, E, E)	List<E>
m	of(E, E, E, E, E, E, E)	List<E>
m	of(E, E, E, E, E, E, E, E)	List<E>
m	of(E, E, E, E, E, E, E, E, E)	List<E>
m	of(E, E, E, E, E, E, E, E, E, E)	List<E>
m	of(E...)	List<E>
m	copyOf(Collection<? extends E>)	List<E>

JDK中的List集合

ArrayList

LinkedList

Vector的用法与ArrayList基本上一致，只不过它是线程安全的，在不需跨线程共享时，用它没有意义，性能还受到影响。

Vector

Stack

LinkedList涵盖了Stack的功能，所以在实际开发中，可以使用LinkedList取代Stack。

List与数组间的转换方法



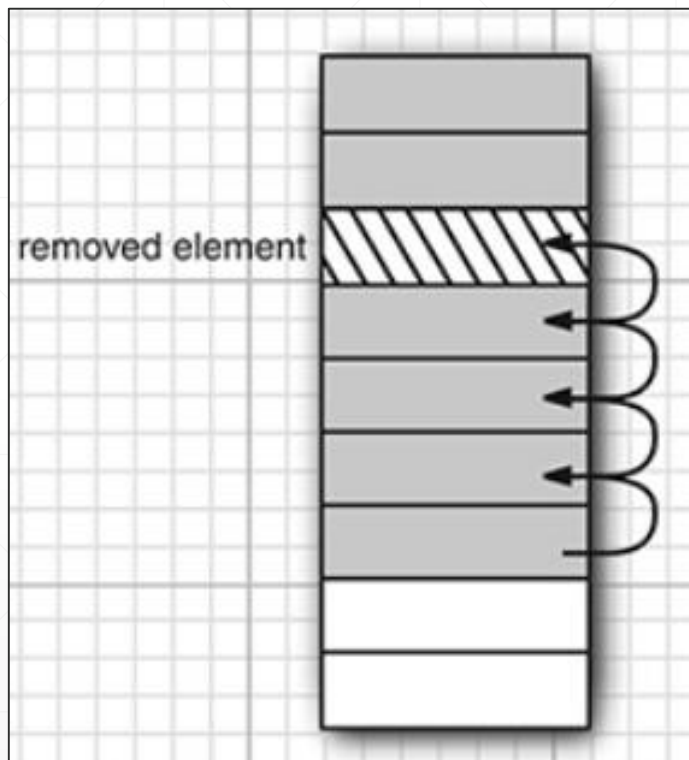
数组与List之间可以相互转换，通常情况下，如果需要增减元素，使用List。如果不需要增减元素，数组是推荐的，因为它通常性能优越。

List是“有序”的，每个元素都有其位置

```
void add(int index, E e);  
E get(int index);  
E remove(int index);  
E set(int index, E element);  
boolean addAll(int index, Collection<? extends E> c);
```

每一个元素有一个索引（`index`），通过这个索引，可以读取或修改List中的元素。这与数组的用法是一致的，毕竟List可以看成是一个可以动态增减元素的“数组”。

以数组形式实现的List，删除一个元素成本很高

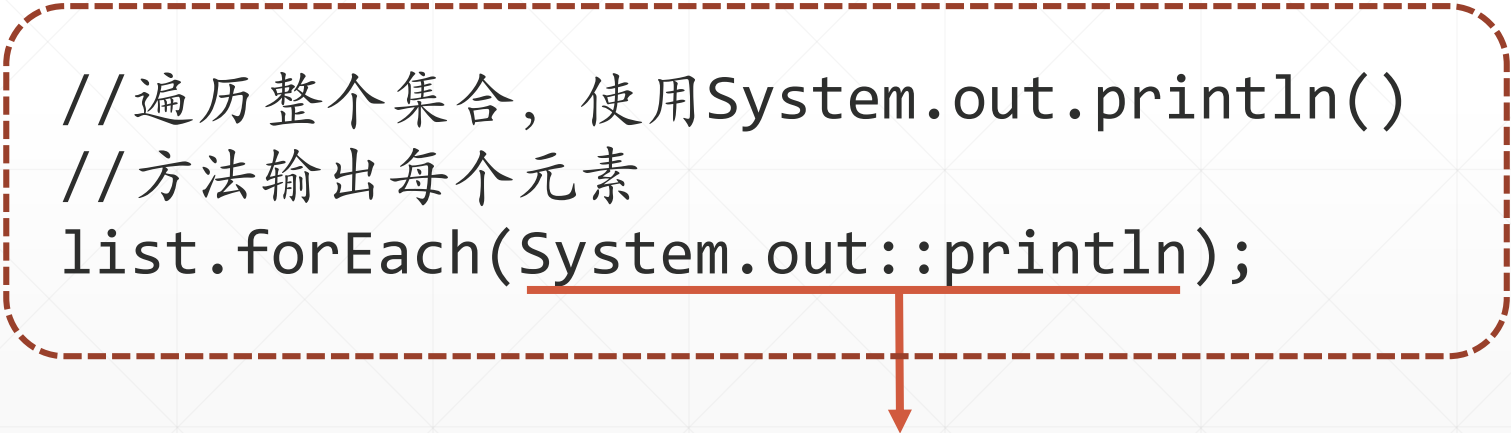


如果List内部使用数组来保存数据（比如JDK中的ArrayList），那么，当集合中元素个数比较多时，在集合的前半部分删除元素，成本很高，因为排在被删除元素后面的所有元素，都需要向前移动一个位置。

遍历

List支持for、foreach循环，特别地，支持使用Lambda表达式的forEach()遍历：

```
//遍历整个集合，使用System.out.println()  
//方法输出每个元素  
list.forEach(System.out::println);
```



方法引用

在List中查找元素

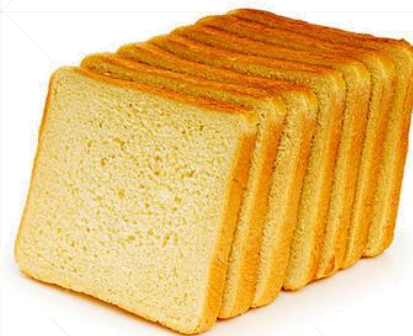
使用indexOf/lastIndex方法在List中正向/反向查找特定的元素，并返回它的位置索引，找不到，返回-1。

```
int indexOf(Object o);  
Int lastIndexOf(Object o);
```



上述方法奏效的前提是：元素具有“判等”功能，也就是说，需要元素重写equals和hashCode方法。

List“切片”



由于List的元素连续存放，并拥有一个位置索引，所以，通过指定一个开始位置和结束位置，可以很方便地将List切分为多段：

```
List<E> subList(int fromIndex, int toIndex);
```

注意：subList返回的List，与原先的List，引用相同的集合对象。所以，修改任何一个，都会影响到另一个。

ArrayList的使用示例

实现List接口的最常用类型是ArrayList，以下示例展示了它的基本用法：

```
List<String> books = new ArrayList<>();  
//向books集合中添加三个元素  
books.add("One book");  
books.add("Two book");  
books.add("Three book");  
System.out.println(books);  
//将新字符串对象插入在第二个位置  
books.add(index: 1, element: "new book");  
for (int i = 0; i < books.size(); i++) {  
    System.out.println(books.get(i));  
}  
//删除第三个元素  
books.remove(index: 2);  
System.out.println(books);  
//判断指定元素在List集合中位置: 输出1, 表明位于第二位  
System.out.println(books.indexOf("new book"));
```

TestArrayList.java

```
//将第二个元素替换成新的字符串对象  
books.set(1, "new book2");  
System.out.println(books);  
//将books集合的第二个元素（包括）到第三个元素（不包括）截取子集合  
var subList : List<String> =books.subList(1,2);  
//输出: [new book2]  
System.out.println(books.subList(1, 2));  
//修改subList的值  
subList.set(0, "新改的值");  
//可以看到原始集合值也改了  
//输出: [One book, 新改的值, Three book]  
System.out.println(books);
```

除了ArrayList，JDK中还有另一个类Vector同样实现了List<E>接口，两者功能基本一样，但ArrayList不是线程安全的（即跨线程修改ArrayList需要加“锁”）。

ArrayList的容量



ArrayList内部使用数组来保存数据，数组本身是固定大小的，这个长度可以在构造ArrayList对象时，通过它的构造方法参数传入，此参数名为initialCapacity（称为“**初始容量**”）。



当持续向ArrayList中插入元素，超过初始容量值时，ArrayList会按照内置算法创建一个更大的数组，将老数组中的内容复制到这个新数组中，从而支持继续向ArrayList中添加新元素。



因此，如果预先知道大致要插入的元素的数量，可以在构建ArrayList时，给它的初始容量指定一个较大的值（或者可以使用ensureCapacity()方法增加当前ArrayList实例的容量值），以便减少频繁创建新数组和复制元素所带来的开销。

反向迭代

List提供一个功能增强了的迭代器，支持对集合元素的**反向**迭代，并且可以使用它在迭代过程中向集合添加新元素：

```
//创建示例集合
String[] books = {
    "One book",
    "Two book"
};
List<String> bookList = new ArrayList<>();
Collections.addAll(bookList, books);
```

示例：TestListIterator.java

```
//获取迭代器对象
ListIterator<String> lit = bookList.listIterator();
System.out.println("先正向迭代");
while (lit.hasNext()) {
    //可以在迭代过程中添加新元素
    lit.add("|");
    System.out.println(lit.next());
}
System.out.println("再反向迭代");
while(lit.hasPrevious()) {
    System.out.println(lit.previous());
}
```

动手动脑



当List中的元素支持判等操作时，可以使用List<E>的以下两个方法查找元素：

```
int indexOf(Object o)        //正向查找元素，返回其位置索引  
int lastIndexOf(Object o)   //反向查找元素，返回其位置索引
```

请编写一个自定义类，创建若干个对象，然后把它们加到ArrayList中，编写代码验证以上特性，并回答以下问题：



indexOf()方法要起作用，你写的那个自定义类必须满足什么条件才行？你可以通过查看ArrayList的indexOf()方法源码，找到这个问题的答案。

固定大小的ArrayList

- 定义于Arrays类内部的ArrayList类，是一个固定长度的List集合，只能遍历访问，不能增加和删除元素，否则，会引发UnsupportedOperationException。
- 可以使用以下代码创建一个固定大小的List

```
List fixedList = Arrays.asList("One book" , "Two book");
```

示例FixedSizeList.java

使用固定大小的ArrayList

```
List<String> fixedList = Arrays.asList(  
    "One book" ,  
    "Two book");  
//获取fixedList的实现类, 将输出Arrays$ArrayList  
System.out.println(fixedList.getClass());  
//遍历fixedList的集合元素  
for (String s : fixedList) {  
    System.out.println(s);  
}  
//试图增加、删除元素都将引发UnsupportedOperationException异常  
fixedList.add("Three book");  
fixedList.remove(0: "One book");
```



Arrays.asList()返回的List也叫ArrayList, 但它与前面介绍过的ArrayList是两个**不同**的类, 这里的ArrayList是Arrays类的内部类, 与前面PPT所介绍的ArrayList, 两者根本不是一回事。

另一种非常常用的创建固定大小List的方法

```
var fixedList2 : List<Integer> = List.of(1, 2, 3);  
//输出: class java.util.ImmutableCollections$ListN  
System.out.println(fixedList2.getClass());  
//输出: [1, 2, 3]  
System.out.println(fixedList2);
```



还有第三种创建固定大小的集合的方法，就是前面课程介绍过的使用`Collections.unmodifiableCollection()`方法，如果你忘记了，请复习一下。

小结



本讲以ArrayList为例，介绍了List的基本使用方法。



JDK中的ArrayList内部以数组方式保存数据，具有非常好的按位置随机访问的特性，但对于频繁插入与删除元素的场合，性能不佳。